OptiGrid

Energy Intelligence Platform

Architectural Requirements

Version 1.0

July 30, 2026

Team Coreflow

COS 301 - Capstone Project

University of Pretoria

Contents

1 Introduction **2**

2 Architectural Requirements **2**
- 2.1 Architectural Patterns . . . 2
- 2.2 Design Patterns . . . 3
- 2.3 Constraints . . . 3
- 2.4 Architectural Diagram . . . 5
- 2.5 Mapping Quality Requirements to Architectural Decisions . . . 6

3 Technology Requirements **7**

4 API Contracts **9**
- 4.1 Authentication . . . 9
- 4.2 User Access Control . . . 11
- 4.3 Buildings . . . 14
- 4.4 Preferences . . . 22
- 4.5 Analytics . . . 23
- 4.6 Sensors . . . 24
- 4.7 Thresholds . . . 27
- 4.8 Support / Contact . . . 31
- 4.9 Telemetry . . . 32
- 4.10 Anomalies . . . 33

5 Deployment **37**
- 5.1 Deployment Requirements . . . 37
  - 5.1.1 Live, Accessible System . . . 37
  - 5.1.2 Environment Parity . . . 37
  - 5.1.3 Infrastructure as Code (IaC) / Containerisation . . . 37
  - 5.1.4 Secrets Management . . . 37
  - 5.1.5 Rollback Strategy . . . 37
- 5.2 Deployment Diagrams . . . 37
  - 5.2.1 CI/CD Pipeline Diagram . . . 38

6 NFR Traceability Matrix **38**

7 Deployment Diagram **40**

1 Introduction

This document outlines the architectural patterns, design choices, system constraints, and the technology stack utilized to keep the platform scalable and secure. The architecture is intentionally designed to support the high throughput of IoT telemetry data while maintaining a responsive and accessible user interface for facility managers.

2 Architectural Requirements

2.1 Architectural Patterns

OptiGrid uses a hybrid approach, combining **Microservices** and **Event-Driven** patterns to ensure the system stays available and scales well under heavy loads.

- **Microservices Architecture:**
  - *Definition:* The system is broken down into smaller, independent services based on what they do (like Data Ingestion, Analytics, and the Dashboard UI), rather than existing as a single monolithic application.
  - *Justification:* This is necessary because ingesting IoT telemetry data handles significantly more traffic than basic dashboard API queries. Keeping these separate means the ingestion containers can be scaled up during heavy data spikes without paying for extra frontend resources or slowing down the dashboard. It also lets the Python analytics engine run entirely separately from the Node.js backend, allowing for language-specific optimizations.
- **Event-Driven Architecture (EDA):**
  - *Definition:* Different parts of the system communicate asynchronously by sending events to a message broker, rather than relying on direct service-to-service calls.
  - *Justification:* If thousands of IoT meters try to send data at the exact same time, normal API calls would back up and probably crash the database. Using a Redis queue acts as a buffer. The ingestion service drops incoming data onto the queue, which lets the database write it at a safe, steady pace. This helps prevent data loss even if traffic spikes unexpectedly.
- **Layered Architecture:**
  - *Definition:* The system logic is split into 4 strictly separated presentation, access, services and databse layers. Thus it is a four-tiered layered architecture
  - *Justification:* This keeps the codebase highly organized. For example, changing the user interface will not interfere with how data is ingested at the database level, which makes maintaining and updating the system much easier over time.

2.2 Design Patterns

Several standard design patterns are used in the codebase to keep the system maintainable, modular, and reliable.

- **Model-View-Controller (MVC):**
  - *Definition:* Separates the application logic into three interconnected parts to separate internal information from how it is presented to the user.
  - *Justification:* This is used extensively in the backend services and loosely mirrored in the React frontend. It separates the raw data models from the dashboard views using controllers. This decoupling makes the code significantly easier to test and keeps different concerns clearly separated during development.
- **Singleton Pattern:**
  - *Definition:* Restricts a class so it can only be instantiated once globally.
  - *Justification:* This is used for the database connection pools (PostgreSQL and InfluxDB). Creating a brand new connection for every single request in a busy IoT system would cause massive delays and exhaust the available connections. A Singleton makes sure a single, persistent connection pool is reused efficiently across all incoming requests.
- **Strategy Pattern:**
  - *Definition:* A behavioral pattern that lets you select a specific algorithm at runtime from a family of algorithms.
  - *Justification:* This is used in the Python Analytics engine to pick the right predictive model. The system relies on Optuna to automatically test and choose the best forecasting strategy based on a building's specific energy profile, rather than hardcoding a single, rigid algorithm that might not fit every scenario.

2.3 Constraints

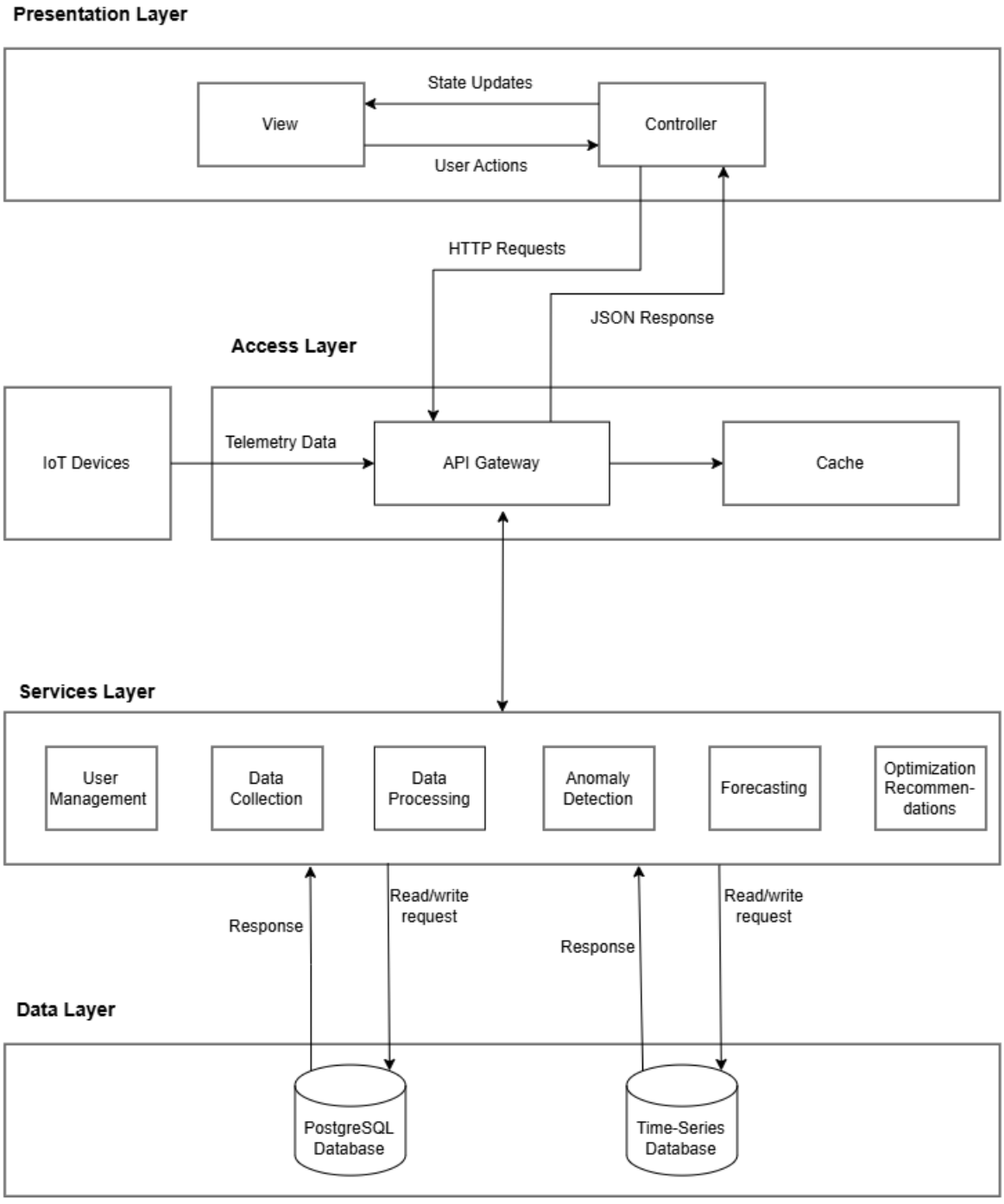
The architectural choices are shaped by a few strict limitations:

- **Financial & Infrastructure Constraints:** The project operates under a strict R5000 budget provided by EPI-USE. *Impact:* The project cannot use paid enterprise software or expensive proprietary platforms. The system must stick to open-source tools and use cost-effective cloud options (like AWS Free Tier) to remain financially viable throughout its lifecycle.
- **Hardware & Data Constraints:** Due to a lack of physical access to real world building smart grids, the system is constrained to supporting simulated telemetry data. *Impact:* The ingestion API has to be flexible enough to take standard JSON payloads over regular HTTPS. It will not be tied to specific hardware protocols; this will make it significantly easier to integrate real-world sensor hardware later on without rewriting the ingestion logic.

- **Deployment Constraints:** The system has to be fully containerised and deployed through an automated CI/CD pipeline, with no manual interventions allowed. *Impact:* Docker must be used for all services. All configuration settings are handled dynamically through `.env` variables so that the development, staging, and production environments are exactly the same without needing to alter source code.
- **Latency & Performance Constraints:** Real-time dashboards must reflect data updates in under 2 seconds. *Impact:* A regular relational database cannot handle the heavy write speeds and fast read queries needed for IoT data. The system is forced to use a dedicated Time-Series Database (InfluxDB) that is expressly built for this kind of high-throughput workload.
- **Security & Regulatory Constraints:** The system handles potentially sensitive building operational data and must adhere strictly to the Protection of Personal Information Act (POPIA). *Impact:* Data has to be encrypted when stored, and strict Role-Based Access Control (RBAC) is needed so users only see their own buildings. The system also has to keep comprehensive audit logs for compliance.

2.4 Architectural Diagram

Below is the technology-neutral structural blueprint of OptiGrid, illustrating the separation of concerns and data flow.



2.5 Mapping Quality Requirements to Architectural Decisions

This table maps the quantified Non-Functional Requirements (from the SRS) directly to the architectural mechanisms that guarantee their fulfilment.

Quality Requirement (from SRS)	**Architectural Mechanism & Justification**
Performance: Respond to requests within 2 seconds for 95% of requests.	**Time-Series Database & Pre-aggregation:** Using InfluxDB for telemetry makes querying date ranges very fast. Background cron jobs are also used to pre-calculate data into 15-minute and hourly summaries. This means the dashboard only has to load small datasets instead of calculating millions of rows on the fly, keeping load times under a second.
Scalability: Support up to 200% workload increase with $< 10\%$ performance drop.	**Microservices + Docker Auto-scaling:** Because the Ingestion Service runs in its own lightweight Docker container, the cloud host can easily spin up extra copies of it when traffic spikes, spreading the load out horizontally and preventing bottlenecking.
Security: AES-256 encryption and multi-factor authentication enforcement.	**Centralised Authentication + TLS 1.3 + JWT:** The authentication layer serves as a strict gateway. All outside traffic is forced to use HTTPS (TLS 1.3), and short-lived JSON Web Tokens (JWTs) are used to safely verify user permissions on every request without storing session data on the server.
Reliability: 99.9% uptime and recover from critical failures < 5 minutes.	**Message Broker (Redis Queue) Buffering:** If the main database goes down, the ingestion API will not lose data. Incoming telemetry waits safely in the Redis queue until the database comes back online, preventing data loss while the system automatically recovers.
Usability: Core tasks completed in <5 min, 85% user satisfaction.	**Dashboard-First MVC UI Design:** Using React along with a robust state management system allows the UI to update instantly without reloading the page. A standard set of design tokens is also used to keep navigation clear and contrast ratios highly accessible for all users.
Maintainability: New features or bug fixes deployable within 2 hours, 80% test coverage.	**Automated Pipelines & Separation of Concerns:** Because the layers are strictly separated and MVC is used, the code is much easier to unit test. The CI/CD pipeline runs these tests automatically, ensuring code is not merged unless it meets the strict 80% coverage rule.

3 Technology Requirements

The following technology stack was selected to balance handling lots of data, running machine learning predictions, and building the UI quickly.

Component	Selected Technology	Justification
Frontend	**React.js**	React is great for building interactive, data-heavy dashboards. Its virtual DOM is fast enough to update complex graphs in real time without lagging. It also has a huge community, making it easy to drop in charting libraries (like Recharts or Chart.js) for the energy graphs.
Backend API	**Node.js (Express)**	The main challenge for OptiGrid's backend is handling thousands of small API requests at the same time. Node.js uses an event loop that does not block other processes, making it much better at handling high volumes of concurrent connections than traditional multi-threaded languages.
Relational Database	**PostgreSQL (Supabase)**	This is only used for system metadata (like user profiles, building configs, and permissions). PostgreSQL is reliable, handles complex table joins well, and ensures data stays strictly isolated between different tenants.
Time-Series Database	**InfluxDB**	Trying to store millions of telemetry data points in a standard SQL database usually ruins indexing performance over time. InfluxDB is specifically designed for time-stamped data, allowing for fast writes and easy automatic data downsampling.

Component	Selected Technology	Justification
Analytics Engine	**Python (scikit-learn, Pandas, Optuna)**	Python is the standard choice for data science. Using libraries like scikit-learn, Pandas, and Optuna allows the system to efficiently predict future demand based on historical data. Putting this in its own Python microservice allows the use of the best machine learning tools without bloating the Node.js backend.
Containerisation	**Docker**	Docker solves the "it works on my machine" problem. It ensures that the code runs exactly the same way on a developer's laptop, in the development environment, and on the AWS cloud servers.
Hosting	**AWS (Backend), Vercel (Frontend)**	AWS allows us to host the backend services on a live platform, it makes it easy for us to deploy our system since we use Docker. Vercel hosts our frontend and speaks to AWS for any requests to fetch data. Vercel creates pre-production environments so we can see how our app looks with code changes before pushing to the production environment.

4 API Contracts

4.1 Authentication

1. **signup**
   - POST
   - **Request:**

     ```
     {
         email: string,
         password: string,
         name: string
     }
     ```

     Example:

     ```
     {
         "email": "user@example.com",
         "password": "SecurePass123!",
         "name": "John Doe"
     }
     ```

   - **Response:** (201 Created)

     ```
     {
         "message": "User created successfully",
         "user": {
             "userId": "123e4567-e89b-12d3-a456-426614174000",
             "email": "user@example.com",
             "firstName": "John",
             "lastName": "Doe"
         }
     }
     ```

   - **Error Responses:**

     ```
     400 Bad Request
     {
         "error_code": "INVALID_INPUT",
         "message": "Invalid email or password format"
     }

     500 Internal Server Error
     {
         "error_code": "SERVER_ERROR",
         "message": "Internal server error"
     }
     ```

   - **Effect:**
     - Creates a new user.

2. **login**

   - POST
   - **Request:**

     ```
     {
         email: string,
         password: string
     }
     ```

     Example:

     ```
     {
         "email": "user@example.com",
         "password": "SecurePass123!"
     }
     ```

   - **Response:** (200 OK)

     ```
     {
         "message": "Login successful",
         "user": {
             "userId": "123e4567-e89b-12d3-a456-426614174000",
             "email": "user@example.com",
             "firstName": "John",
             "lastName": "Doe",
             "token": "eyJhbGciOiJIUzI1NiIsInR..."
         }
     }
     ```

   - **Error Responses:**

     ```
     401 Unauthorized
     {
         "error_code": "UNAUTHORIZED",
         "message": "Invalid credentials provided"
     }

     500 Internal Server Error
     {
         "error_code": "SERVER_ERROR",
         "message": "Internal server error"
     }
     ```

   - **Effect:**
     - Authenticates a user with their email and password.
     - Returns user information along with a JWT token which expires after some time.

4.2 User Access Control

1. **getViewers**
   - GET
   - **Request:** None

     Example: `GET /auth/viewers`
   - **Response:** (200 OK)

```
{
    "data": [
        {
            "userId": "123e4567-e89b-12d3-a456-426614174000",
            "email": "viewer@example.com",
            "firstName": "Jane",
            "lastName": "Smith",
            "roleType": "VIEWER",
            "buildingIds": ["uuid-1", "uuid-2"]
        }
    ]
}
```

   - **Error Responses:**

```
403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

   - **Effect:**
     - Fetches all users with the VIEWER role and their assigned building IDs.
     - Requires ADMIN access.
2. **getManagers**
   - GET
   - **Request:** None

     Example: `GET /auth/managers`
   - **Response:** (200 OK)

```
{
```

```
    "data": [
        {
            "userId": "123e4567-e89b-12d3-a456-426614174000",
            "email": "manager@example.com",
            "firstName": "Mark",
            "lastName": "Johnson",
            "roleType": "BUILDING_MANAGER",
            "buildingIds": ["uuid-1"]
        }
    ]
}
```

- **Error Responses:**

```
403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Fetches all users with the BUILDING_MANAGER role and their assigned building IDs.
  - Requires ADMIN access.

3. **assignManager**
   - POST
   - **Request:**

```
{
    userId: string,
    buildingId: string
}
```

Example:

```
{
    "userId": "123e4567-e89b-12d3-a456-426614174000",
    "buildingId": "888e4567-e89b-12d3-a456-426614174000"
}
```

   - **Response:** (200 OK)

```
{
```

```
    "success": true,
    "message": "Manager assigned to building successfully"
}
```

- **Error Responses:**

```
400 Bad Request
{
    "error_code": "INVALID_INPUT",
    "message": "Invalid request body"
}

403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Assigns a building manager to a specific building.
  - Requires ADMIN access.

4. **removeManager**
   - DELETE
   - **Request:**

```
{
    userId: string,
    buildingId: string
}
```

Example:

```
{
    "userId": "123e4567-e89b-12d3-a456-426614174000",
    "buildingId": "888e4567-e89b-12d3-a456-426614174000"
}
```

- **Response:** (200 OK)

```
{
    "success": true,
    "message": "Manager removed from building successfully"
}
```

- **Error Responses:**

```
403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

404 Not Found
{
    "error_code": "NOT_FOUND",
    "message": "Manager or building not found"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Removes a manager's access from a specific building.
  - Requires ADMIN access.

4.3 Buildings

1. **getAllBuildings**
   - GET
   - **Request:** Query Parameters: `lifecycle_state: string (optional)`

     Example: `GET /admin?lifecycle_state=ACTIVE`
   - **Response:** (200 OK)

```
{
    "status": "success",
    "data": [
        {
            "building_id": "123e4567-e89b-12d3-a456-426614174000",
            "building_name": "Main Campus",
            "building_type": "EDUCATIONAL",
            "lifecycle_state": "ACTIVE"
        }
    ]
}
```

   - **Error Responses:**

```
403 Forbidden
{
```

```
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Fetches all buildings that are in the database.
  - Requires ADMIN access.

2. **getManagerBuildings**
   - GET
   - **Request:** None

     Example: `GET /manager`
   - **Response:** (200 OK)

```
{
    "status": "success",
    "data": [
        {
            "building_id": "123e4567-e89b-12d3-a456-426614174000",
            "building_name": "Science Block",
            "todays_usage": 152.4
        }
    ]
}
```

   - **Error Responses:**

```
403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

   - **Effect:**
     - Fetches all buildings assigned to the currently authenticated manager.

3. **createBuilding**

   - POST
   - **Request:**

```
Headers:
    idempotency-key: string
Body:
{
    building_name: string,
    building_type: string,
    square_footage?: number,
    timezone?: string,
    max_occupancy?: number,
    physical_address?: string,
    latitude?: number,
    longitude?: number,
    geohash?: string
}
```

     Example:

```
POST /api/buildings
Headers: { "idempotency-key": "550e8400-e29b-41d4-a716-446655440000" }
{
    "building_name": "Corporate Headquarters",
    "building_type": "OFFICE",
    "square_footage": 75000,
    "physical_address": "123 Main Street"
}
```

   - **Response:** (201 Created)

```
{
    "status": "success",
    "data": {
        "id": "123e4567-e89b-12d3-a456-426614174000",
        "building_name": "Corporate Headquarters",
        "building_type": "OFFICE"
    }
}
```

   - **Error Responses:**

```
400 Bad Request
{
    "error_code": "INVALID_INPUT",
    "message": "Invalid request body"
}

403 Forbidden
```

```
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Creates a new building with the provided details.
  - Uses an idempotency key to prevent duplicate requests.
  - The building is created in 'PROVISIONING' state by default.

4. **getBuildingDetails**
   - GET
   - **Request:** Path Parameters: `building_id: string`

     Example: `GET /api/buildings/123e4567-e89b-12d3-a456-426614174000`
   - **Response:** (200 OK)

```
{
    "status": "success",
    "data": {
        "building_id": "123e4567-e89b-12d3-a456-426614174000",
        "building_name": "Corporate Headquarters",
        "timezone": "Africa/Johannesburg"
    }
}
```

   - **Error Responses:**

```
403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

404 Not Found
{
    "error_code": "NOT_FOUND",
    "message": "Building not found"
}

500 Internal Server Error
{
```

```
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Returns all relevant details for a single building including.
  - Only accessible if the authenticated user has access to it.

5. **getBuildingEnergyConsumption**
   - GET
   - **Request:** Path Parameters: `building_id: string`
     Query Parameters: `time_range: string (e.g., '7d', '30d')`

     Example: `GET /api/buildings/.../energy-consumption?time_range=7d`
   - **Response:** (200 OK)

```
{
    "status": "success",
    "data": {
        "building_id": "123e4567-e89b-12d3-a456-426614174000",
        "total_kwh": 1234.56,
        "average_daily_kwh": 41.15,
        "cost_per_kwh": 2.5
    }
}
```

   - **Error Responses:**

```
400 Bad Request
{
    "error_code": "INVALID_QUERY",
    "message": "Invalid time_range specified"
}

403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

404 Not Found
{
    "error_code": "NOT_FOUND",
    "message": "Building not found"
}

500 Internal Server Error
{
```

```
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Returns energy consumption metrics for a single building over a selected time range.

6. **updateBuilding**
   - PATCH
   - **Request:**

```
Path Parameters: building_id: string
Body:
{
    building_name?: string,
    building_type?: string,
    square_footage?: number,
    timezone?: string,
    max_occupancy?: number,
    physical_address?: string
}
```

Example:

```
PATCH /api/buildings/123e4567-e89b-12d3-a456-426614174000
{
    "building_name": "Renamed Corporate Office"
}
```

- **Response:** (200 OK)

```
{
    "status": "success",
    "data": { ...updated_building_details... }
}
```

- **Error Responses:**

```
400 Bad Request
{
    "error_code": "INVALID_INPUT",
    "message": "Invalid request body"
}

403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}
```

```
404 Not Found
{
    "error_code": "NOT_FOUND",
    "message": "Building not found"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Updates specific properties of a building.
  - Requires ADMIN or MANAGER access.

7. **deleteBuilding**
   - DELETE
   - **Request:** Path Parameters: `building_id: string`

     Example: `DELETE /api/buildings/123e4567-e89b-12d3-a456-426614174000`
   - **Response:** (200 OK)

```
{
    "status": "success",
    "message": "Building successfully deleted"
}
```

   - **Error Responses:**

```
403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

404 Not Found
{
    "error_code": "NOT_FOUND",
    "message": "Building not found"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

   - **Effect:**

- Removes a building from the database.
- Requires ADMIN access.

8. **compareBuildings**
   - POST
   - **Request:**

     ```
     Headers: idempotency-key: string
     Query Parameters:
         buildingID_1: string
         buildingID_2: string
         time_range: string (e.g., '30d')
     ```

     Example: `POST /api/buildings/compare?buildingID_1=uuid1&buildingID_2=uuid2&ti`

   - **Response:** (200 OK)

     ```
     {
         "status": "success",
         "data": {
             "building_a": { "energy_consumption": 1500, "cost": 3000 },
             "building_b": { "energy_consumption": 1200, "cost": 2400 },
             "comparison": { ... }
         }
     }
     ```

   - **Error Responses:**

     ```
     400 Bad Request
     {
         "error_code": "INVALID_INPUT",
         "message": "Invalid query parameters"
     }

     403 Forbidden
     {
         "error_code": "FORBIDDEN",
         "message": "User does not have access"
     }

     500 Internal Server Error
     {
         "error_code": "SERVER_ERROR",
         "message": "Internal server error"
     }
     ```

   - **Effect:**
     - Compares energy consumption and performance metrics between two chosen buildings for a given time range.

4.4 Preferences

1. **getUserTheme**
   - GET
   - **Request:** None
     Example: `GET /api/preferences/theme`
   - **Response:** (200 OK)

     ```
     {
         "theme": "dark"
     }
     ```
   - **Error Responses:**

     ```
     401 Unauthorized
     {
         "error_code": "UNAUTHORIZED",
         "message": "Authentication required"
     }

     500 Internal Server Error
     {
         "error_code": "SERVER_ERROR",
         "message": "Internal server error"
     }
     ```
   - **Effect:**
     - Retrieves the current user's theme preference (light, dark, or system).
2. **updateUserTheme**
   - PUT
   - **Request:**

     ```
     {
         theme: "light" | "dark" | "system"
     }
     ```

     Example: `PUT /api/preferences/theme`
   - **Response:** `200 OK (Preference updated)`
   - **Error Responses:**

     ```
     400 Bad Request
     {
         "error_code": "INVALID_INPUT",
         "message": "Invalid theme preference"
     }

     401 Unauthorized
     ```

```
{
    "error_code": "UNAUTHORIZED",
    "message": "Authentication required"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Updates the current user's theme preference in the database.

4.5 Analytics

1. **getForecast**
   - POST
   - **Request:**

     ```
     Path Parameters: building_id: string
     Body:
     {
         horizon_days: number,
         granularity: string
     }
     ```

     Example: `POST /api/analytics/forecast/...`
   - **Response:** (200 OK)

     ```
     {
         "historical": [ ... ],
         "forecast": [ ... ],
         "summary": { ... }
     }
     ```
   - **Error Responses:**

     ```
     400 Bad Request
     {
         "error_code": "INVALID_INPUT",
         "message": "Invalid request body"
     }

     403 Forbidden
     {
         "error_code": "FORBIDDEN",
         "message": "User does not have access"
     }
     ```

```
404 Not Found
{
    "error_code": "NOT_FOUND",
    "message": "Building not found"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Calculates an energy usage forecast graph for a specific building based on historical data.

4.6 Sensors

1. **handleSensorTelemetry**
   - POST
   - **Request:**

```
{
    building_id: string,
    sensor_id: string,
    usage: number,
    timestamp?: string
}
```

   - **Response:** (201 Created)

```
{
    "status": "success"
}
```

   - **Error Responses:**

```
400 Bad Request
{
    "error_code": "INVALID_INPUT",
    "message": "Invalid request body"
}

403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}
```

```
500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Submit telemetry data to sensors securely.

2. **listSensors**
   - GET
   - **Request:** Query Parameters: `building_id: string`
     Example: `GET /api/sensors?building_id=...`
   - **Response:** (200 OK)

```
[
    {
        "id": "sensor-001",
        "mac_address": "AA:BB:CC:DD:EE:FF",
        "status": "Active"
    }
]
```

   - **Error Responses:**

```
403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

404 Not Found
{
    "error_code": "NOT_FOUND",
    "message": "Building not found"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

   - **Effect:**
     - Returns every sensor registered to the given building.
     - Allowed for any authenticated user with access to the building.

3. **registerSensor**

- POST
- **Request:**

```
{
    building_id: string,
    mac_address: string,
    sensor_type: string,
    unit: string,
    location_zone: string,
    status: string
}
```

- **Response:** `201 Created (Sensor registered successfully)`
- **Error Responses:**

```
400 Bad Request
{
    "error_code": "INVALID_INPUT",
    "message": "Invalid request body"
}

403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Registers a new sensor for the specified building.
  - Requires ADMIN and MANAGER access.

4. **deleteSensor**
   - DELETE
   - **Request:** Path Parameters: `sensor_id: string`
     Example: `DELETE /api/sensors/sensor-001`
   - **Response:** `200 OK (Sensor deleted successfully)`
   - **Error Responses:**

```
403 Forbidden
{
    "error_code": "FORBIDDEN",
```

```
    "message": "User does not have access"
}

404 Not Found
{
    "error_code": "NOT_FOUND",
    "message": "Sensor not found"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Deletes the sensor from its building.
  - Requires ADMIN and MANAGER access.

4.7 Thresholds

1. **createThreshold**
   - POST
   - **Request:**

```
POST /api/thresholds
{
    "building_id": string,
    "metric_type": string,
    "unit": string,
    "upper_limit_kw": number,
    "lower_limit_kw": number,
    "allowed_spike_percentage": number,
    "use_z_score": boolean,
    "z_score_threshold": number
}
```

   - **Response:** (201 Created)

```
{
    "status": "success",
    "data": {
        "threshold_id": string,
        "building_id": string
    }
}
```

   - **Error Responses:**

```
403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Creates an alert threshold for a building.

2. **getThresholds**
   - GET
   - **Request:** Path Parameters: `buildingId: string`

     `GET /api/thresholds/building/{buildingId}`
   - **Response:** (200 OK)

```
{
    "status": "success",
    "data": [
        {
            "threshold_id": string,
            "metric_type": string,
            "upper_limit_kw": number
        }
    ]
}
```

   - **Error Responses:**

```
403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

   - **Effect:**
     - Fetches a list of active thresholds related to a specific building.

3. **updateThreshold**
   - PATCH
   - **Request:** Path Parameters: `id: string`

     ```
     PATCH /api/thresholds/{id}
     {
         "upper_limit_kw": number,
         "lower_limit_kw": number,
         "allowed_spike_percentage": number,
         "use_z_score": boolean,
         "z_score_threshold": number,
         "is_active": boolean,
         "muted_until": string (date-time)
     }
     ```

   - **Response:** (200 OK)

     ```
     {
         "status": "success",
         "data": { ... }
     }
     ```

   - **Error Responses:**

     ```
     403 Forbidden
     {
         "error_code": "FORBIDDEN",
         "message": "User does not have access"
     }

     404 Not Found
     {
         "error_code": "NOT_FOUND",
         "message": "Threshold not found"
     }

     500 Internal Server Error
     {
         "error_code": "SERVER_ERROR",
         "message": "Internal server error"
     }
     ```

   - **Effect:**
     - Modifies some fields of a threshold (e.g., upper limit, z-score usage).
4. **deleteThreshold**
   - DELETE
   - **Request:** Path Parameters: `id: string`

```
DELETE /api/thresholds/{id}
```

- **Response:** (200 OK)

```
{
    "status": "success",
    "message": "Threshold deleted"
}
```

- **Error Responses:**

```
403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

404 Not Found
{
    "error_code": "NOT_FOUND",
    "message": "Threshold not found"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Removes a threshold and syncs the changes.

5. **muteThreshold**

- PATCH
- **Request:** Path Parameters: `id: string`

```
PATCH /api/thresholds/{id}/mute
{
    "muted_until": string (date-time)
}
```

- **Response:** (200 OK)

```
{
    "status": "success",
    "data": {
        "muted_until": string (date-time)
    }
}
```

- **Error Responses:**

```
403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

404 Not Found
{
    "error_code": "NOT_FOUND",
    "message": "Threshold not found"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Sets or clears a muting period for a threshold so alerts are not generated.

4.8 Support / Contact

1. **handleSubmit**
   - POST
   - **Request:**

```
POST /api/contact
{
    "inquiryType": "Building",
    "subject": "Cannot create a building",
    "message": "I cannot create a building and need help."
}
```

   - **Response:** (200 OK)

```
{
    "success": true,
    "message": "Ticket Received",
    "id": "5b304c4f-1234-5f56-4321-8dcb76cd1123"
}
```

   - **Error Responses:**

```
400 Bad Request
{
    "success": false,
    "error": "Failed to send: API key is missing or invalid"
}
```

- **Effect:**
  - Sends a contact/support request email through Resend.

4.9 Telemetry

1. **ingestTelemetry**
   - POST
   - **Request:**

     ```
     POST /api/telemetry/ingest
     ```

   - **Response:** `200 OK`
   - **Error Responses:**

     ```
     400 Bad Request
     {
         "error_code": "INVALID_INPUT",
         "message": "Invalid telemetry payload"
     }

     500 Internal Server Error
     {
         "error_code": "SERVER_ERROR",
         "message": "Internal server error"
     }
     ```

   - **Effect:**
     - Used by physical circuits and python emulator to ingest data.
2. **getLivePortfolioTelemetry**
   - GET
   - **Request:**

     ```
     GET /api/telemetry/live
     ```

   - **Response:** `200 OK`
   - **Error Responses:**

     ```
     403 Forbidden
     {
         "error_code": "FORBIDDEN",
         "message": "User does not have access"
     }

     500 Internal Server Error
     {
         "error_code": "SERVER_ERROR",
         "message": "Internal server error"
     ```

```
}
```

- **Effect:**
  - Used by frontend real-time dashboard for aggregate views.

3. **streamTelemetry**
   - GET
   - **Request:** Path Parameters: `building_id: string`

     `GET /api/telemetry/stream/{building_id}`
   - **Response:** `200 OK (Stream)`
   - **Error Responses:**

```
403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

404 Not Found
{
    "error_code": "NOT_FOUND",
    "message": "Building not found"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

   - **Effect:**
     - Used by frontend dashboard to stream telemetry.

4.10 Anomalies

1. **getAnomalies**
   - GET
   - **Request:** Path Parameters: `buildingId: string`
     Query Parameters: `skip`, `take`, `severity`, `status`, `startDate`, `endDate`

     `GET /api/anomalies/building/{buildingId}`
   - **Response:** (200 OK)

```
{
    "status": "success",
    "data": [
```

```
        {
            "anomaly_id": string,
            "severity_level": string,
            "status": string,
            "detected_timestamp": string
        }
    ],
    "meta": {
        "total": integer,
        "skip": integer,
        "take": integer
    }
}
```

- **Error Responses:**

```
403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access to this building"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Fetches a list of anomalies associated with a specific building.

2. **updateAnomalyStatus**

- PATCH
- **Request:** Path Parameters: `id: string`

```
PATCH /api/anomalies/{id}/status
{
    "status": "Resolved"
}
```

- **Response:** (200 OK)

```
{
    "status": "success",
    "data": {
        "anomaly_id": string,
        "status": string,
        "resolved_timestamp": string
    }
}
```

- **Error Responses:**

```
400 Bad Request
{
    "error_code": "INVALID_INPUT",
    "message": "Invalid status value provided"
}

403 Forbidden
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access to this building"
}

404 Not Found
{
    "error_code": "NOT_FOUND",
    "message": "Anomaly not found"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Modifies the status of an anomaly, updates the resolved_timestamp if marked as 'Resolved'.

3. **getAnomalyContext**
   - GET
   - **Request:** Path Parameters: `id: string`

```
GET /api/anomalies/{id}/context
```

   - **Response:** (200 OK)

```
{
    "status": "success",
    "data": {
        "anomaly_id": string,
        "sensor": { ... },
        "threshold": { ... }
    }
}
```

   - **Error Responses:**

```
403 Forbidden
```

```
{
    "error_code": "FORBIDDEN",
    "message": "User does not have access to this building"
}

404 Not Found
{
    "error_code": "NOT_FOUND",
    "message": "Anomaly not found"
}

500 Internal Server Error
{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}
```

- **Effect:**
  - Fetches context for a specific anomaly, including sensor data and the threshold that were breached.

5 Deployment

Deployment refers to publishing and running the software system in an accessible environment so that users, clients, and evaluators can interact with it without requiring local installation.

5.1 Deployment Requirements

5.1.1 Live, Accessible System

Optigrid is deployed and accessible via the link in the root README. The frontend is accesible via the public url which connects to the backend hosted on AWS whichis only accessible to team members.

5.1.2 Environment Parity

The system distinguishes between development and production environments. The main branch automatically deploys to at least one non-local environment using the Vercel configurations. Development is managed locally via Docker commands and Supabase CLI or Supabase web editor to replicate prodcution database. Production: Automatically deployed when develop is merged into main branch.

5.1.3 Infrastructure as Code (IaC) / Containerisation

The complete system is deployable through Docker and Docker Compose, ensuring deployments are reproducible across environments. No manual deployment steps ("click-ops") are required.

Containerisation: All backend and frontend services are containerised using optimised Dokcerfiles available in their respective folders. IAC: AWS resources are provisioned using Terraform and are controlled.

5.1.4 Secrets Management

All environment variables are put in a .gitnore file. They are stored securely using Github Secrets and Vercel/AWS Parameter Store.

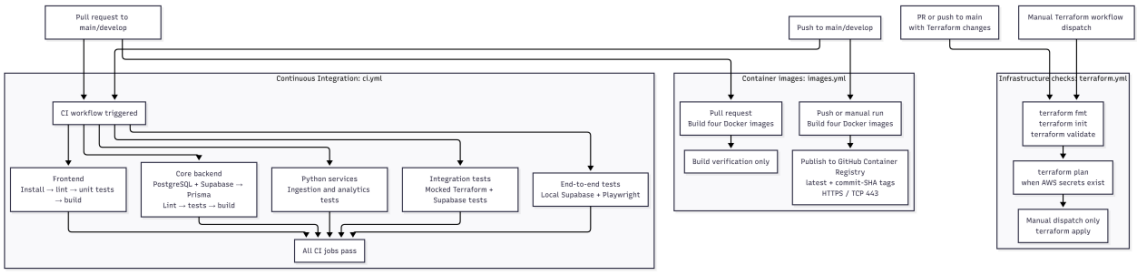
5.1.5 Rollback Strategy

To ensure rapid recovery from deployment failure we implement a multi-tiered rollback poliicies: 1. Container Image Tagging: Docker images in production are tagged with release commit hash alongside latest. If a deployment fails, the docker containers are revereted to the previously working version ensuring rapid recovery. 2. Frontend Instant Rollback: Vercel deployment allow instant 1-click rollbacks to the last knonw stable frontend deployment if the frontend fails with new code.

5.2 Deployment Diagrams

The following diagrams illustrate where the system is deployed and how software changes move through the deployment pipeline.

5.2.1 CI/CD Pipeline Diagram



6 NFR Traceability Matrix

ID	Quantified requirement	Tactic in SAS	Test / Tool	Status
A01	Available 24/7, excluding maintenance	Health checks	Uptime Monitor	PASS
A02	Redundant components fail smoothly	Redundancy/managed restart	Docker CLI	PASS
A03	Graceful degradation under network latency	Cache fallback	Docker CLI	PASS
A04	Zero-downtime deployment	Rolling restarts	Docker CLI	PASS
R01	Recover from critical failures within 5 min	Orchestration restart	Docker CLI	PASS
R02	99.9% uptime under load	Auto-scaling	k6	PASS
R03	Database connection recovery	Connection pool limits	Docker CLI	FAIL
R04	Resilience to malformed telemetry data	Input validation	Automated API Tests	PASS
SEC01	SAST & Dependencies	Automated dependency updates	npm audit	FAIL
SEC02	Configuration & sec headers	Security headers	OWASP ZAP	PASS
SEC03	Operational & Abuse (Rate Limiting)	API Rate Limiting	k6	PASS
SEC04	Authentication & AES-256	JWT & AES-256 encryption	Automated API Tests	PASS
PERF-01	Performance Smoke (1 VU, 30s)	Request handling	k6	PASS
PERF-02	Performance Concurrency (50 VUs)	Load balancing	k6	PASS
PERF-03	Performance Load (500 VUs)	Load balancing	k6	PASS
SC01	+200% workload / ≤10% latency degradation	Scalable Infrastructure	k6 + Docker Compose	FAIL
SC02	Horizontal scaling effectiveness	Scalable Infrastructure	k6 + Docker Compose	PASS
SC03	Dashboard isolation while ingestion scales	Scalable Infrastructure	k6 + Docker Compose	FAIL
SC04	Burst buffering & accepted-point persistence	Scalable Infrastructure	k6 + Docker Compose	PASS
SC05	Load traffic-triggered	Scalable Infrastructure	k6 + Docker	BLOCKED

7 Deployment Diagram